

FACTORY MANAGEMENT SYSTEM

A modular CLI-based Python application

Submitted by

Harsh Vijay

Submitted to

Dr. Priyadharshini B.

Registration No.

26BCE10808

Project Overview

What is it?

Beginner-friendly CLI system for small-scale factories.

Core idea

Digitize stock, categories and machine audit records.

Technology

Python 3.x, OOP and standard libraries.

Run

```
python main.py
```

Problem Statement & Objectives

Problem

Manual logs can cause lost records and inaccurate efficiency tracking.

Objectives

Digitize basic operations and persist inventory between runs.

Scope & Target Users

Factory Supervisor

Products, stock updates, audits.

Inventory Manager

Stock and categories.

Maintenance Staff

Audit results and machine faults.

Functional Requirements

FR-01 Add

Capture product ID, name, category, stock and price.

FR-02 View

Display all inventory.

FR-03 Categories

Track unique categories.

FR-04 Update

Add manufactured units.

FR-05 Audit

Simulate machine status and efficiency.

FR-06 Persist

Save/load factory_stock.txt.

High-Level Features

Inventory

Add and update products.

Categories

Unique category set.

Machine Audit

Four predefined machines.

Efficiency

Working/total percentage.

Persistence

Flat-file storage.

Modularity

Focused Python modules.

System Architecture

CLI

main.py controls interaction.

Logic

inventory.py + audit.py.

Models

Product + Machine.

Data

data_manager.py + text file.

- main.py -> inventory.py / audit.py -> data_manager.py -> factory_stock.txt
- models.py provides Product and Machine objects.

System Workflow

1 Start

Load saved inventory.

2 Menu

Select one of six actions.

3 Process

Delegate to module.

4 Persist/Report

Save or print audit.

5 Continue

Repeat or exit.

- Start -> Load Data -> Show Menu -> Process Request -> Save/Report -> Continue or Exit

Module / Component Design

main.py

CLI and orchestration.

inventory.py

Product operations.

data_manager.py

Persistence.

audit.py

Machine audit.

models.py

Domain classes.

Class / Component Diagram

Product

p_id, name, stock, price; display_info().

Machine

m_id, name, is_working.

- Product: p_id, name, stock, price | Machine: m_id, name, is_working

Data Storage Model

In-memory

Dictionary for inventory + set for categories.

Persistent

factory_stock.txt stores ID, name, stock, price.

ER note

No relational database is used in the current implementation.

Implementation Details

Python

Core language.

OOP

Product and Machine classes.

Libraries

os, random, math, datetime.

Audit logic

Efficiency = $\text{floor}(\text{working}/\text{total} \times 100)$.

Screens / Results

CLI Menu

Six operations are exposed.

Audit Report

Machine status and efficiency are printed.

Persistence

Saved inventory reloads on restart.

- Representative CLI output based on the implemented menu and audit logic; machine status is randomized at runtime.

Program Execution Screenshots

Actual console output showing product registration, inventory, machine audit and save/exit.

```

8:38 Wed, Sep 20
SMALL FACTORY MANAGEMENT SYSTEM
=====
[1] Add New Product
[2] View All Inventory
[3] View Registered Categories
[4] Update Stock After Production
[5] Run Floor Machine Audit
[6] Save and Exit
=====
Select an option (1-6): 1

NEW PRODUCT ENTRY
=====
Enter Product ID: 1001
Enter Product Name: Cotton
Enter Category (e.g., Raw, Assembly): Raw
Enter Initial Stock Units: 500
Enter Unit Price (INR): 2000

Status: Product 'Cotton' registered successfully.
=====
SMALL FACTORY MANAGEMENT SYSTEM
=====
[1] Add New Product
[2] View All Inventory
[3] View Registered Categories
[4] Update Stock After Production
[5] Run Floor Machine Audit
[6] Save and Exit
=====
Select an option (1-6): 2

CURRENT FACTORY STOCK
=====
[ID: 1001] Cotton | Available Stock: 500 | Price: rs 2000.0
=====
SMALL FACTORY MANAGEMENT SYSTEM
=====
[1] Add New Product

```

```

8:39 Wed, Sep 20
[ID: 1001] Cotton | Available Stock: 420 | Price: rs 2000.0
=====
SMALL FACTORY MANAGEMENT SYSTEM
=====
[1] Add New Product
[2] View All Inventory
[3] View Registered Categories
[4] Update Stock After Production
[5] Run Floor Machine Audit
[6] Save and Exit
=====
Select an option (1-6): 5

DAILY AUDIT REPORT: 2026-09-20
=====
[ALERT] Assembly Conveyor (M-01) -> FAULT DETECTED
[OK] Automated Paint Rig (M-02) -> OPERATIONAL
[OK] Hydraulic Press (M-03) -> OPERATIONAL
[OK] Packaging Line (M-04) -> OPERATIONAL

Machines Online : 3 / 4
Overall Efficiency : 75%
=====
SMALL FACTORY MANAGEMENT SYSTEM
=====
[1] Add New Product
[2] View All Inventory
[3] View Registered Categories
[4] Update Stock After Production
[5] Run Floor Machine Audit
[6] Save and Exit
=====
Select an option (1-6): 6

All changes saved, Closing system. Goodbye!
=====
[Program finished]

```

Testing Approach

Registration

Add and view product.

Duplicate ID

Reject existing ID.

Stock update

Verify quantity changes.

Persistence

Restart and verify records.

Audit

Run repeatedly and verify output.

Input validation

Check invalid numeric input.

Challenges Faced & Design Decisions

Data loss

Solved with local persistence.

Machine visibility

Solved with audit module.

Maintainability

Solved through modular files and OOP.

Learnings & Key Takeaways

Python

Core syntax, functions and libraries.

OOP

Entity-based design.

File handling

Save and load structured records.

Modularity

Separation of responsibilities.

Problem solving

Map real workflow to software.

Conclusion

Current version

Lightweight CLI system for small-scale operations.

Future scope

Database, GUI/web UI, authentication, sensors and analytics.

Thank You

Questions & Discussion

- The current implementation is intentionally lightweight and can be extended with a database, GUI/web interface, authentication and real sensor data.